QTM 151 - Quiz 1

Submit as an HTML file

Quiz is open 11:30am to 12:20pm

Print your name below

In [47]:
```
# Write your answer here

print("Nick Huberty")
```

```
Nick Huberty
```

This quiz is open book

- You can use the lecture notes
- You will get partial credit for attempting the questions
- To get full credit, the code should work as intended
- You should NOT communicate with other students

Print the following message:

"I will abide by Emory's code of conduct"

In [48]:
```
# Write your answer here:
print("I will abide by Emory's code of conduct")
```

```
I will abide by Emory's code of conduct
```

Import the libraries "numpy", "matplotlib.pyplot", and "pandas"

In [49]:
```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
```

1. IF/ELIF/ELSE statements

- In the beginning of the code cell below, two lists are created, where element 0 is one of two random choices:

```
list_p = ["Introduction"/"Intro", "to", "Python"]
list_r = ["Introduction"/"Intro", "to", "R"]
```

- Create a new variable `keyword = "Introduction"`
- Create an if/elif/else block that tests whether `keyword` is part of either list. Depending on the outcome (belongs to list_p, belongs to list_r, doesn't belong to either list) print an appropriate message to the screen.

HINT: If the creation of list_p and list_r is confusing, add print statements to view the two lists and run the cell 3-4 times to see some of the possible situations

In [84]:
```
### create lists DO NOT CHANGE
str_choices = [["Introduction", "Intro"], ["Intro", "Introduction"], ["Intro", "Int
str_indices = np.random.choice([0, 1, 2])
start_str = str_choices[str_indices]

list_p = [start_str[0], "to", "Python"]
list_r = [start_str[1], "to", "R"]

### Your solution below

keyword = "Introduction"
if keyword in list_p:
    print(f"{keyword} belongs to list p.")
elif keyword in list_r:
    print(f"{keyword} belongs to list r.")
else:
    print(f"{keyword} does not belong to either list.")
```

```
Introduction belongs to list r.
```

2. Generate a vector of values and plot it

- Below, two lists are defined containing values of x_k and y_k

```
coord_x = [1,4, 0, -1, 20, 40]
coord_y = [2,2, 0, 10, 0, 1]
```

- Create a vector (**vector always means numpy array**) `hypotenuse_sqr` with elements h_k defined by the formula

$$h_k = x_k^2 + y_k^2$$

- You **should not** define it manually: so "hypoteneuse_sqr = np.array([5, 20, 0, 101, 400, 1601])" is not an acceptable answer
- Produce a **histogram** with the elements of `hypotenuse_sqr` on the horizontal axis
- Label the axes and the title
- Change the color to "orange", "green" or "purple" (you choose)
- Make sure the plot appears in an output cell

Hint: Most mathematical operations are defined for Numpy arrays, which allows you to avoid loops

Hint: type "help(plt.hist)" to see which argument is used to change color

In [97]:
```
coord_x = [1, 4, 0, -1, 20, 40]
coord_y = [2, 2, 0, 10, 0, 1]

### Your solution below
x_vals = np.linspace(1, 4, 0, -1, 20, 40)
y_vals = np.linspace(2, 2, 0, 10, 0, 1)
hypotenuse_sqr = x_vals ** 2 + y_vals **2
plt.hist(hypotenuse_sqr, bins=20, color='orange')
plt.title("Histogram of hypotenuse squares")
plt.xlabel("x coordinates")
plt.ylabel("y coordinates")
plt.show()
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[97], line 5
      2 coord_y = [2, 2, 0, 10, 0, 1]
      4 ### Your solution below
----> 5 x_vals = np.linspace(1, 4, 0, -1, 20, 40)
      6 y_vals = np.linspace(2, 2, 0, 10, 0, 1)
      7 hypotenuse_sqr = x_vals ** 2 + y_vals **2

File c:\Users\pompo\anaconda3\Lib\site-packages\numpy\_core\function_base.py:143, in
linspace(start, stop, num, endpoint, retstep, dtype, axis, device)
    141     integer_dtype = False
    142 else:
--> 143     integer_dtype = _nx.issubdtype(dtype, _nx.integer)
    145 # Use `dtype=type(dt)` to enforce a floating point evaluation:
    146 delta = np.subtract(stop, start, dtype=type(dt))

File c:\Users\pompo\anaconda3\Lib\site-packages\numpy\_core\numerictypes.py:530, in
issubdtype(arg1, arg2)
    473 r"""
    474 Returns True if first argument is a typecode lower/equal in type hierarchy.
    475
   (...)
    527
    528 """
    529 if not issubclass_(arg1, generic):
--> 530     arg1 = dtype(arg1).type
    531 if not issubclass_(arg2, generic):
    532     arg2 = dtype(arg2).type

TypeError: Cannot interpret '40' as a data type
```

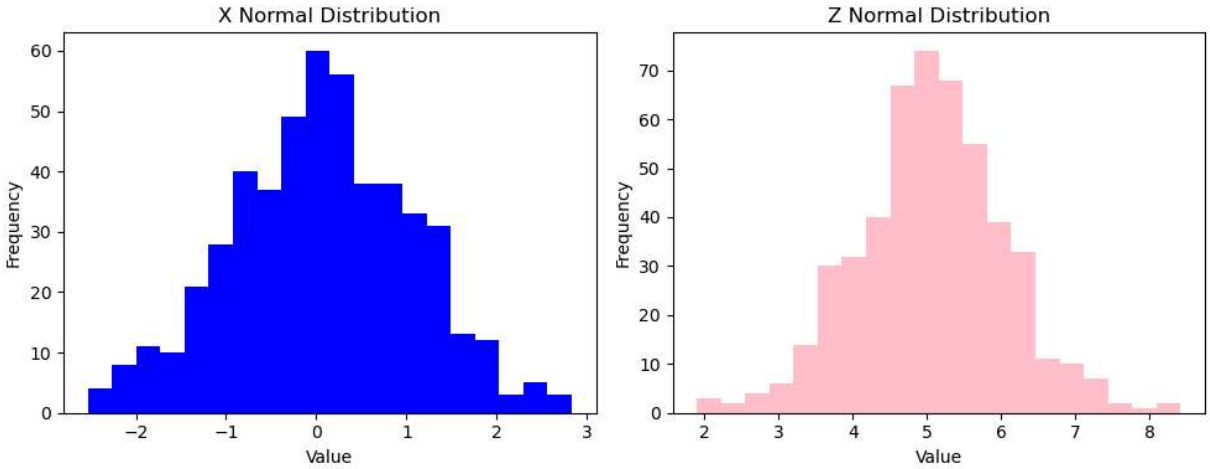
3. Histograms of random variables

- Let $n = 500$
- Generate "vec_x" from a normal distribution with (loc = 0,scale = 1,size = n)
- Generate "vex_z" from a normal distribution with (loc = 5,scale = 1,size = n)
- Plot two **histograms** on a single row using the `plt.subplots` syntax
  - Label the axes and title
  - Each graph should have its axes labeled, and should have their own title

Hint: If you can't remember how subplots works, generate two individual plots for partial credit

In [98]:
```
n = 500

### Your solution below:
vec_x = np.random.normal(loc=0, scale=1, size=500)
```

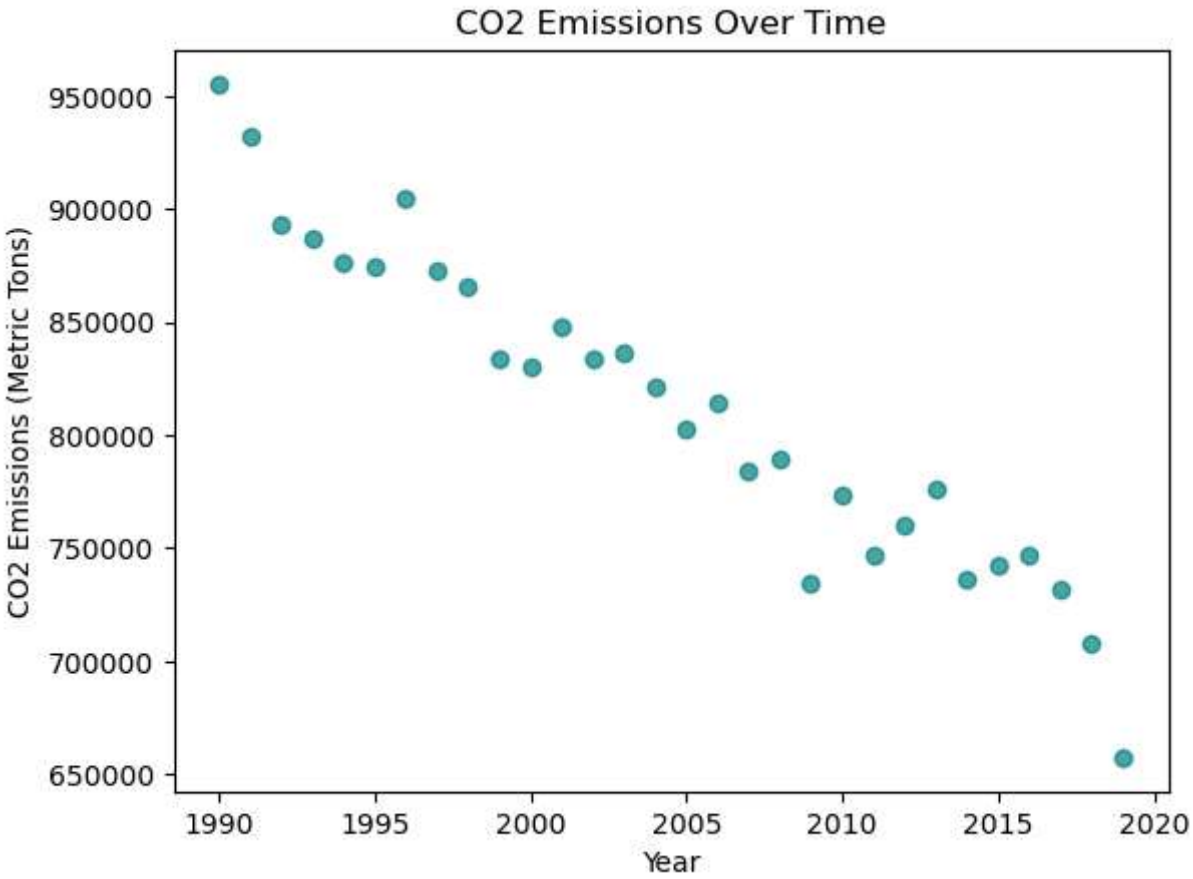
```
vex_z = np.random.normal(loc=5, scale=1, size=500)
fig, axes = plt.subplots(1, 2, figsize=(10, 4))

axes[0].hist(vec_x, bins=20, color='blue')
axes[0].set_title("X Normal Distribution")
axes[0].set_xlabel("Value")
axes[0].set_ylabel("Frequency")

axes[1].hist(vex_z, bins=20, color='pink')
axes[1].set_title("Z Normal Distribution")
axes[1].set_xlabel("Value")
axes[1].set_ylabel("Frequency")

plt.tight_layout()
plt.show()
```



4. Combine information across datasets

- Read the dataset "/data_quiz_1/germany_co2_emissions.csv" and store it as "data_germany"
- View the dataset, and see what variables it contains
- Create a **scatter plot** of CO2 emissions with the following characteristics
  - The yearly variable on the x-axis.
  - Germany's CO2 emissions on the y-axis.
  - Label the axes

In [99]:
```
### Your solution below:
file_path = r"C:\Users\pompo\Downloads\quiz1_section1\quiz1_section1\data_quiz_1\ge
data_germany = pd.read_csv(file_path)
plt.scatter(data_germany['year'], data_germany['total_emissions'], color='teal', al
plt.title("CO2 Emissions Over Time")
plt.xlabel("Year")
plt.ylabel("CO2 Emissions (Metric Tons)")
plt.show()
```



5. Loop through dfferent variables and plot

- Read the dataset "data_quiz_1/wdi_ageprop_2020.csv"
- Create a list of variable names with the following elements:

```
list_varnames = ["percent_ages0to14","percent_ages15to64"]
```

- Run a for loop that computes the mean for each variable and print it on screen.

Hint: Computing the mean of a column/variable from a Pandas dataset is just like computing the mean of a Numpy array

In [107…
```
### Your solution below:
file_path = r"C:\Users\pompo\Downloads\quiz1_section1\quiz1_section1\data_quiz_1\wd
ages = pd.read_csv(file_path)
list_varnames = ["percent_ages0to14", "percent_ages15to64"]
for col in list_varnames:
    if col in list_varnames:
        mean_val = ages[col].mean()
        print(f"The mean of {col} is {mean_val:.2f}")
    else:
        print(f"Column '{col}' not found in DataFrame.")
```

```
The mean of percent_ages0to14 is 26.91
The mean of percent_ages15to64 is 63.68
```

6. Run loop through different datasets and compute statistics

- Create a list with the ".csv" dataset names of two countries of your choice. For example

  `["data_quiz_1/name1.csv","data_quiz_1/name2.csv"]`

- Create an empty list called `list_emissions`
- Run a for-loop over the list of dataset names. Inside the loop:
  - Read the corresponding ".csv" dataset.
  - Compute the mean of "total_emissions" for each respective country, and store it as an element of `list_emissions`
- After the loop, print `list_emissions`

HINT: Use `.append()` (See Lecture 5-supplement or Lecture 7)

In [108…
```
### Your solution below:
csv_files = [
    r"C:\Users\pompo\Downloads\quiz1_section1\quiz1_section1\data_quiz_1\korea_rep_
    r"C:\Users\pompo\Downloads\quiz1_section1\quiz1_section1\data_quiz_1\japan_co2_
]
list_emissions = []

for file in csv_files:
    df = pd.read_csv(file)
    if 'total_emissions' in df.columns:
        mean_val = df['total_emissions'].mean()
        list_emissions.append(mean_val)
    else:
```

```
        list_emissions.append(None)

print("List of mean Emissions values from files:")
print(list_emissions)
```

```
List of mean Emissions values from files:
[np.float64(287606.4942740235), np.float64(954347.021200521)]
```